Finding Files and Directories

Now, we are comfortable creating, modifying, moving, and deleting files and directories. We should cover a beneficial concept that can make or break it during an engagement or in our day-to-day tasks as a `System Administrator` or `Penetration Tester`, known as `enumeration`. This section will cover how to search for particular files and directories utilizing CMD, why enumerating system files and directories are vital, and provide an essential list of what to look out for while enumerating the system.

Searching With CMD

Using Where

Finding Files and Directories

```
C:\Users\student\Desktop>where calc.exe

C:\Windows\System32\calc.exe

C:\Users\student\Desktop>where bio.txt

INFO: Could not find files for the given pattern(s).
```

Above, we can see two different tries using the `where` command. First, we searched for `calc.exe`, and it completed showing us the path for calc.exe. This command worked because the system32 folder is in our environment variable path, so the `where` command can look through those folders automatically.

The second attempt we see failed. This is because we are searching for a file that does not exist within that environment path. It is located within our user directory. So we need to specify the path to search in, and to ensure we dig through all directories within that path, we can use the `/R` switch.

Recursive Where

Finding Files and Directories

```
C:\Users\student\Desktop>where /R C:\Users\student\ bio.txt

C:\Users\student\Downloads\bio.txt
```

Above, we searched recursively, looking for bio.txt. The file was found in the `C:\Users\student\Downloads\` folder. The `/R` switch forced the `where` command to search through every folder in the student user directory hive. On top of looking for files, we can also search wildcards for specific strings, file types, and more. Below is an example of searching for the `csv` file type within the student directory.

Using Wildcards

Finding Files and Directories

```
C:\Users\student\Desktop>where /R C:\Users\student\ *.csv

C:\Users\student\AppData\Local\live-hosts.csv
```

Integrated Terminal

┌─[us-academy-1]─[10.10.14.248]─[htb-ac-819475@htb-qcrcj6h43i]─[~]
└──╼ [★]$

We used `where` to give us an idea of how to search for files and applications on the host. Now, let us talk about `Find`. Find is used to search for text strings or their absence within a file or files. You can also use `find` against the console's output or another command. Where `find` is limited, however, is its capability to utilize wildcard patterns in its matching. The example below will show us a simple search with Find against the not-password.txt file.

Basic Find

Finding Files and Directories

```
C:\Users\student\Desktop> find "password" "C:\Users\student\not-passwords.txt"
```

We can modify the way `find` searches using several switches. The `/V` modifier can change our search from a matching clause to a `Not` clause. So, for example, if we use `/V` with the search string password against a file, it will show us any line that does not have the specified string. We can also use the `/N` switch to display line numbers for us and the `/I` display to ignore case sensitivity. In the example below, we use all of the modifiers to show us any lines that do not match the string `IP Address` while asking it to display line numbers and ignore the case of the string.

Find Modifiers

Finding Files and Directories

```
C:\Users\student\Desktop> find /N /I /V "IP Address" example.txt
```

For quick searches, find is easy to use, but it could be more robust in how it can search. However, if we need something more specific, `findstr` is what we need. The `findstr` command is similar to `find` in that it searches through files but for patterns instead. It will look for anything matching a pattern, regex value, wildcards, and more. Think of it as find2.0. For those familiar with Linux, `findstr` is closer to `grep`.

Findstr

Finding Files and Directories

```
C:\Users\student\Desktop> findstr
```

Evaluating and Sorting Files

We have seen how to work with, search for certain files and search for strings inside files. Additionally, we have also learned how to create and modify files. Now let us discuss a few options to evaluate those files and compare them against each other. The comp, fc, and sort commands are how we will accomplish this.

`Comp` will check each byte within two files looking for differences and then displays where they start. By default, the differences are shown in a decimal format. We can use the `/A` modifier if we want to see the differences in ASCII format. The `/L` modifier can also provide us with the line numbers.

Compare

Finding Files and Directories

```
C:\Users\student\Desktop> comp .\file-1.md .\file-2.md
```

Integrated Terminal

```
Files compare OK
```

Above, we see the comparison come back OK. The files are the same. We can use this as an easy way to check if any scripts, executables, or critical files have been modified. Below we have output from a file that's been changed.

Comparing Different Files

Finding Files and Directories

```
PS C:\htb> echo a > .\file-1.md
PS C:\Users\MTanaka\Desktop> echo a > .\file-2.md
PS C:\Users\MTanaka\Desktop> comp .\file-1.md .\file-2.md /A
Comparing .\file-1.md and .\file-2.md...
Files compare OK
<SNIP>
PS C:\Users\MTanaka\Desktop> echo b > .\file-2.md
PS C:\Users\MTanaka\Desktop> comp .\file-1.md .\file-2.md /A
Comparing .\file-1.md and .\file-2.md...
Compare error at OFFSET 2
file1 = a
file2 = b
```

We used echo to ensure the strings differed and then reran the comparison. Notice how our output changed, and using the /A modifier, we are seeing the character difference between the two files now. `Comp` is a simple but effective tool. Now let us look at `FC` for a minute. `FC` differs in that it will show you which lines are different, not just an individual character (`/A`) or byte that is different on each line. FC has quite a few more options than Comp has, so be sure to look at the help output to ensure you are using it in the manner you want.

FC Help

Finding Files and Directories

```
C:\htb> fc.exe /?

Compares two files or sets of files and displays the differences between
them

FC [/A] [/C] [/L] [/LBn] [/N] [/OFF[LINE]] [/T] [/U] [/W] [/nnnn]
   [drive1:][path1]filename1 [drive2:][path2]filename2
FC /B [drive1:][path1]filename1 [drive2:][path2]filename2

  /A         Displays only first and last lines for each set of differences.
  /B         Performs a binary comparison.
  /C         Disregards the case of letters.
  /L         Compares files as ASCII text.
  /LBn       Sets the maximum consecutive mismatches to the specified
             number of lines.
  /N         Displays the line numbers on an ASCII comparison.
  /OFF[LINE] Do not skip files with offline attribute set.
  /T         Does not expand tabs to spaces.
  /U         Compare files as UNICODE text files.
  /W         Compresses white space (tabs and spaces) for comparison.
  /nnnn      Specifies the number of consecutive lines that must match
             after a mismatch.
  [drive1:][path1]filename1
             Specifies the first file or set of files to compare.
  [drive2:][path2]filename2
             Specifies the second file or set of files to compare.
```

When FC performs its inspection, it is case-sensitive and cares more than just a byte-for-byte comparison. Below we will use a few

Integrated Terminal

and the ASCII comparison using the `/N` modifier.

FC

Finding Files and Directories

```
C:\Users\student\Desktop> fc passwords.txt modded.txt /N

Comparing files passwords.txt and MODDED.TXT
***** passwords.txt
    1:  123456
    2:  password
***** MODDED.TXT
    1:  123456
    2:
    3:  password
*****

***** passwords.txt
    5:  12345
    6:  qwerty
***** MODDED.TXT
    6:  12345
    7:  Just something extra to show functionality. Did it see the space inserted above?
    8:  qwerty
*****
```

The output from FC is much easier to interpret and gives us a bit more clarity about the differences between the files. When comparing files such as text files, spreadsheets, or lists, it is prudent to sort them first to ensure the data on each string is the same. Otherwise, every line will be different, and our comparison will not help. Let us look at `sort` now to help us with that. With `Sort`, we can receive input from the console, pipeline, or a file, sort it and send the results to the console or into a file or another command. It is relatively simple to use and often will be used in conjunction with pipeline operators such as `|`, `<`, and `>`. We can give it a try now by feeding the contents of the `file` file to sort.

Sort

Finding Files and Directories

```
C:\Users\student\Desktop> type .\file-1.md
a
b
d
h
w
a
q
h
g

C:\Users\MTanaka\Desktop> sort.exe .\file-1.md /O .\sort-1.md
C:\Users\MTanaka\Desktop> type .\sort-1.md

a
a
b
d
g
h
h
q
w
```

Integrated Terminal

list of letters, sorted them in alphabetical order, and wrote them to the new file. It can get more complex when working with larger datasets, but the basic usage is still the same. If we wanted `sort` only to return unique entries, we could also use the /unique modifier. Notice the first two entries in the `sort-1.md` file . Let us try using unique and see what happens.

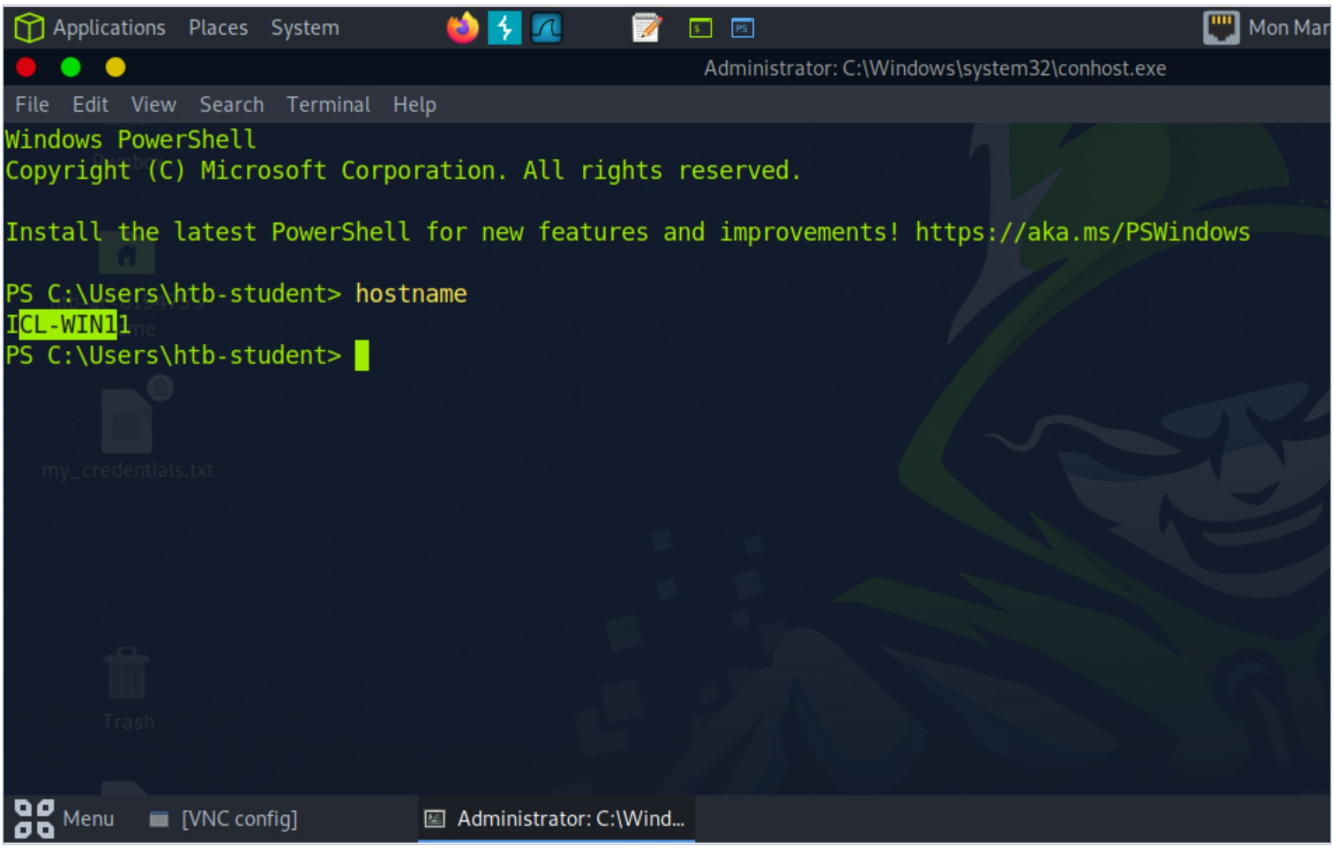
unique

Finding Files and Directories

```
C:\htb> type .\sort-1.md

a
a
b
d
g
h
h
q
w

PS C:\Users\MTanaka\Desktop> sort.exe .\sort-1.md /unique

a
b
d
g
h
q
w
```

Notice how we have fewer overall results now. This is because `sort` did not write duplicate entries from the file to the console.

Finding files and directories, sorting datasets, and comparing files are all essential skills we should have in our arsenal. Next, we will discuss Environment variables and what they provide us as a cmd-prompt user.



Full Screen | Terminate | Reset

Connected to htb-qcrcj6h43i:1 (htb-ac-819475)

Life Left: 117m +

Questions

Cheat Sheet

Download VPN Connection File

Answer the question(s) below to complete this Section and earn cubes!

Target: 10.129.197.103

Life Left: 117 minute(s) + Terminate X

+ 0 What command can be used to search for regular expression strings from command prompt?

Submit your answer here...

Submit Hint

SSH to 10.129.197.103 with user "htb-student" and password "HTB_@cademy_stdnt!"

+ 0 Using the skills acquired in this and previous sections, access the target host and search for the file named 'waldo.txt'. Submit the flag found within the file.

Submit your answer here...

Submit Hint

← Previous Next →

Integrated Terminal

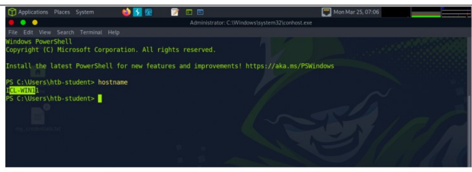
Cheat Sheet

Resources

? Go to Questions

Table of Contents

My Workstation



Integrated Terminal

Interact Terminate Reset Life Left: 117m +

Integrated Terminal